TAREA – COMPARAR LA IMPLEMENTACIÓN DE UN DICCIONARIO DE DATOS FÍSICO EN DIFERENTES SGBD.

Bases de Datos I

Juan David Jovel López
Angie Vanessa Rincón Osorio

Programa: Tecnología en desarrollo de software Virtual

Grupo: G100

Docente: Oralia Cortés Grajales

Institución: Institución Universitaria Pascual Bravo

Fecha de entrega: 25 de octubre de 2026

Justificación de la selección de los sistemas gestores de bases de datos

Para desarrollar la implementación física de la Red Social Estudiantil Pascualina se seleccionaron PostgreSQL, MySQL y MongoDB, con el propósito de comparar dos sistemas gestores relacionales y uno NoSQL orientado a documentos.

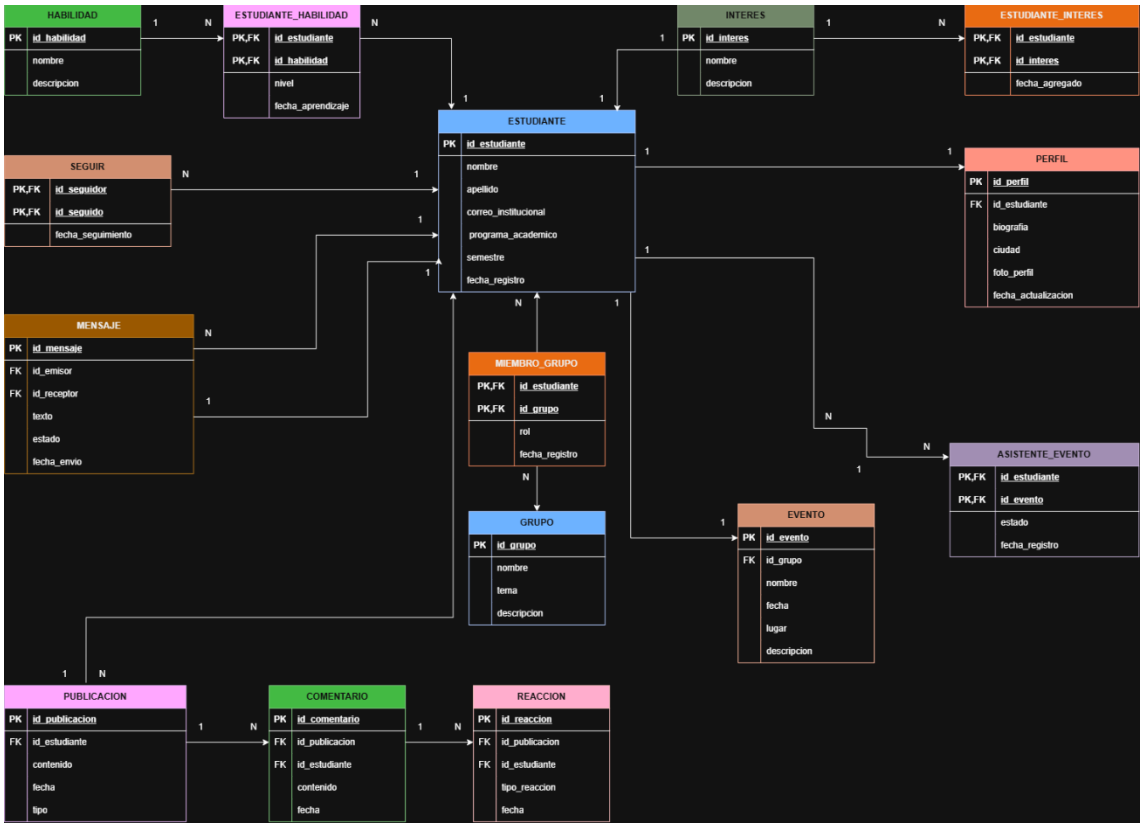
PostgreSQL se seleccionó por sus capacidades para gestionar información relacional, establecer restricciones de integridad y almacenar datos semiestructurados mediante el tipo JSONB. Este último permite complementar las tablas tradicionales con información flexible, sin renunciar a las relaciones y restricciones del modelo relacional.

MySQL se incorporó como segundo gestor relacional para comparar la implementación de las tablas, las claves primarias y foráneas, las restricciones y el manejo de datos JSON. Su tipo de dato JSON permite almacenar documentos y validar que su contenido tenga una estructura JSON válida.

Por su parte, MongoDB se seleccionó como alternativa NoSQL orientada a documentos. Su modelo de almacenamiento utiliza documentos BSON, que admiten estructuras anidadas y campos flexibles. Esto facilita explorar casos de uso en los que la información cambia de estructura o contiene atributos variables, como los datos generados por sensores o los registros de actividad de la plataforma.

La selección permite estudiar las similitudes y diferencias entre los tres sistemas, evaluar la adaptación del modelo lógico a cada tecnología y analizar qué características resultan convenientes según las necesidades de la aplicación. La comparación no busca afirmar que un sistema sea universalmente superior, sino reconocer las ventajas, limitaciones y particularidades de cada alternativa.

Criterio	PostgreSQL	MySQL	MongoDB
Modelo	Relacional (objeto-relacional)	Relacional	Documental (NoSQL)
Datos semiestructurados	JSONB binario con índices GIN y operadores propios (->, ->>, @>)	JSON con índices indirectos mediante columnas generadas	Nativo (BSON)
Integridad referencial	FK, CHECK y UNIQUE completos	FK y CHECK (desde 8.0.16)	Sin FK; validación por esquema
Claves autogeneradas	GENERATED ALWAYS AS IDENTITY	AUTO_INCREMENT	ObjectId
Ajuste al modelo de 15 tablas	Alto	Alto	Bajo (el modelo es muy relacional)



Análisis del modelo lógico

El modelo lógico de la Red Social Estudiantil Pascualina constituye la base para desarrollar la implementación física de la base de datos. Su estructura permite organizar la información relacionada con los estudiantes y las diferentes

actividades que pueden realizar dentro de la plataforma, manteniendo la integridad de los datos y estableciendo relaciones entre las entidades.

El modelo está compuesto por quince tablas que representan los principales componentes de la red social. Cada tabla cuenta con atributos definidos de acuerdo con la información que necesita almacenar, así como claves primarias para identificar los registros y claves foráneas para establecer las relaciones correspondientes entre las entidades.

Entidades principales

Las quince tablas del modelo lógico se pueden agrupar según su función dentro de la aplicación:

- **Gestión de estudiantes y perfiles:** las tablas ESTUDIANTE y PERFIL almacenan la información básica de los usuarios y los datos complementarios de su perfil. La relación entre ambas permite asociar un perfil con un estudiante.
- **Habilidades e intereses:** las tablas HABILIDAD e INTERES almacenan las habilidades y los intereses disponibles en la plataforma. Las tablas intermedias ESTUDIANTE_HABILIDAD y ESTUDIANTE_INTERESES permiten relacionar a los estudiantes con múltiples habilidades e intereses, registrando información adicional como el nivel de habilidad, la fecha de aprendizaje o la fecha en que se agregó un interés.
- **Seguimiento y comunicación:** las tablas SEGUIR y MENSAJE permiten representar las relaciones de seguimiento entre estudiantes y el envío de mensajes privados. En SEGUIR se identifican el estudiante que sigue y el estudiante seguido, mientras que MENSAJE registra el emisor, el receptor, el contenido, el estado y la fecha de envío.
- **Grupos y eventos:** las tablas GRUPO y EVENTO permiten organizar grupos estudiantiles y las actividades asociadas a ellos. Las tablas MIEMBRO_GRUPO y ASISTENTE_EVENTO registran la participación de los estudiantes en grupos y eventos, respectivamente, incluyendo información como el rol del integrante o el estado de inscripción.
- **Publicaciones e interacciones:** las tablas PUBLICACION, COMENTARIO y REACCION permiten almacenar los contenidos

publicados por los estudiantes y las interacciones generadas sobre ellos. Los comentarios y las reacciones se relacionan tanto con la publicación correspondiente como con el estudiante que los realiza.

Relaciones entre las entidades

El modelo utiliza relaciones de uno a uno, uno a muchos y muchos a muchos, de acuerdo con las necesidades de la aplicación.

La relación entre ESTUDIANTE y PERFIL es de uno a uno, debido a que cada estudiante puede tener un perfil asociado y cada perfil pertenece a un único estudiante. Esta relación se representa mediante la clave foránea id_estudiante en la tabla PERFIL, que debe ser única para evitar que un mismo estudiante tenga varios perfiles.

Las relaciones de uno a muchos se presentan, por ejemplo, entre ESTUDIANTE y PUBLICACION, ya que un estudiante puede crear varias publicaciones, y entre PUBLICACION y COMENTARIO, debido a que una publicación puede recibir varios comentarios. También se presentan entre GRUPO y EVENTO, pues un grupo puede tener varios eventos asociados, y entre ESTUDIANTE y MENSAJE, considerando los diferentes mensajes que un estudiante puede enviar o recibir.

Las relaciones de muchos a muchos se resuelven mediante tablas intermedias. ESTUDIANTE_HABILIDAD conecta estudiantes con habilidades; ESTUDIANTE_INTERESES relaciona estudiantes con intereses; MIEMBRO_GRUPO vincula estudiantes con grupos; y ASISTENTE_EVENTO relaciona estudiantes con eventos. En estos casos, las claves primarias compuestas permiten identificar cada asociación mediante la combinación de las claves de las entidades relacionadas.

La tabla SEGUIR también utiliza una clave primaria compuesta por id_seguidor e id_seguido, ambas relacionadas con la tabla ESTUDIANTE. Esta estructura permite representar las relaciones de seguimiento entre usuarios y evita registrar dos veces la misma asociación. Además, se debe impedir que un estudiante se siga a sí mismo.

Claves e integridad de los datos

Las claves primarias permiten identificar de manera única cada registro. En las tablas principales, como ESTUDIANTE, HABILIDAD, INTERES, GRUPO, EVENTO, PUBLICACION, COMENTARIO, MENSAJE y REACCION, se utilizan identificadores propios. En las tablas intermedias se emplean claves primarias compuestas para identificar las relaciones entre entidades.

Las claves foráneas permiten mantener la integridad referencial, evitando que se registren relaciones con estudiantes, publicaciones, grupos o eventos inexistentes. Por ejemplo, un comentario debe estar asociado a una publicación válida y a un estudiante existente, mientras que un evento debe estar vinculado a un grupo registrado.

También se identifican restricciones adicionales necesarias para proteger la calidad de la información, como la unicidad del correo institucional de cada estudiante, la asociación de un único perfil por estudiante y la validación de valores que deban cumplir determinadas condiciones.

Estas reglas son importantes durante la implementación física, ya que deben adaptarse a las herramientas y mecanismos de validación disponibles en cada sistema gestor de bases de datos.

Adaptación del modelo lógico a los sistemas gestores

El modelo lógico se utiliza como referencia común para las tres implementaciones, pero su estructura debe adaptarse a las características de PostgreSQL, MySQL y MongoDB.

En PostgreSQL y MySQL, las entidades se implementan mediante tablas relacionales, utilizando tipos de datos, claves primarias, claves foráneas y restricciones para mantener la integridad de la información. Aunque ambos sistemas comparten el enfoque relacional, pueden presentar diferencias en la definición de ciertos tipos de datos y en la configuración de sus restricciones.

En MongoDB, la implementación se realiza mediante colecciones y documentos. Los identificadores y las referencias permiten relacionar los documentos correspondientes a las distintas entidades. Sin embargo, las relaciones entre documentos no cuentan de forma nativa con el mismo mecanismo de claves

foráneas de los sistemas relacionales, por lo que la consistencia de las referencias debe gestionarse mediante validaciones y lógica de la aplicación.

Además, se contempla el atributo datos_adicionales en la entidad EVENTO para almacenar información semiestructurada. En MySQL puede utilizarse el tipo JSON, mientras que en MongoDB esta información puede representarse mediante un objeto dentro del documento del evento. Este atributo permite registrar datos complementarios de estructura variable sin modificar los atributos principales de la entidad.

En conclusión, el análisis del modelo lógico permite identificar qué información debe almacenarse, cómo se relacionan las entidades y qué reglas deben conservarse durante la implementación. De esta manera, se establece una base común para desarrollar las tres soluciones físicas, respetando las necesidades de la red social y adaptando los mecanismos técnicos a cada sistema gestor.

Diccionario de datos genérico

Tabla 1. ESTUDIANTE

Almacena la información principal de los estudiantes registrados en la red social.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_estudiante	Entero	PK	Obligatorio, autogenerado	Identificador único del estudiante.
nombre	Texto	-	Obligatorio	Nombre del estudiante.
apellido	Texto	-	Obligatorio	Apellido del estudiante.
correo_institucional	Texto	-	Obligatorio, único	Correo institucional del estudiante.
programa_academico	Texto	-	Obligatorio	Programa académico al que pertenece.
semestre	Entero	-	Obligatorio, mayor que cero	Semestre académico que cursa.

fecha_registro	Fecha y hora	-	Obligatorio	Fecha y hora en que se registró.

Tabla 2. PERFIL

Guarda la información complementaria del perfil de cada estudiante.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_perfil	Entero	PK	Obligatorio, autogenerado	Identificador único del perfil.
id_estudiante	Entero	FK	Obligatorio, único	Estudiante dal que pertenece el perfil.
biografia	Texto largo	-	Opcional	Presentación personal del estudiante.
ciudad	Texto	-	Opcional	Ciudad de residencia indicada.
foto_perfil	Texto	-	Opcional	Ruta o URL de la foto de perfil.
fecha_actualizacion	Fecha y hora	-	Opcional	Fecha de la última actualización del perfil.

Se propone que **id_estudiante** sea único en esta tabla para garantizar que un estudiante no tenga varios perfiles. La relación 1:1 del diagrama permite como máximo un perfil por estudiante; no obliga necesariamente a que todos los estudiantes tengan uno.

Tabla 3. HABILIDAD

Contiene las habilidades que los estudiantes pueden registrar en la plataforma.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_habilidad	Entero	PK	Obligatorio, autogenerado	Identificador único de la habilidad.
nombre	Texto	-	Obligatorio	Nombre de la habilidad.

descripcion	Texto largo	-	Opcional	Descripción de la habilidad.

Tabla 4. ESTUDIANTE_HABILIDAD

Relaciona a los estudiantes con las habilidades que poseen. Implementa una relación de muchos a muchos.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_estudiante	Entero	PK, FK	Obligatorio	Estudiante que registra la habilidad.
id_habilidad	Entero	PK, FK	Obligatorio	Habilidad asociada al estudiante.
nivel	Texto	-	Obligatorio	Nivel de dominio de la habilidad.
fecha_aprendizaje	Fecha	-	Opcional	Fecha asociada al aprendizaje de la habilidad.

La clave primaria compuesta por **id_estudiante** e **id_habilidad** evita registrar repetidamente la misma habilidad para un mismo estudiante.

Tabla 5. INTERES

Almacena los interese que pueden asociarse a los estudiantes.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_interes	Entero	PK	Obligatorio, autogenerado	Identificador único del interés.
nombre	Texto	-	Obligatorio	Nombre del interés.
descripcion	Texto largo	-	Opcional	Descripción del interés.

Tabla 6. ESTUDIANTE_INTERES

Relaciona a los estudiantes con sus intereses

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción

id_estudiante	Entero	PK, FK	Obligatorio	Estudiante asociado al interés.
id_interes	Entero	PK, FK	Obligatorio	Interés seleccionado.
fecha_agregado	Fecha y hora	-	Obligatorio	Fecha en que se agregó el interés.

La clave primaria compuesta impide que un mismo interés se asigne más de una vez al mismo estudiante.

Tabla 7. SEGUIR

Registra las relaciones entre estudiantes que se siguen dentro de la red social.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_seguidor	Entero	PK, FK	Obligatorio	Estudiante que inicia el seguimiento.
id_seguido	Entero	PK, FK	Obligatorio	Estudiante al que se sigue.
fecha_seguimiento	Fecha y hora	-	Obligatorio	Fecha en que comenzó el seguimiento.

La clave primaria compuesta evita duplicar el seguimiento entre los mismos estudiantes. Se recomienda impedir que un estudiante se siga a sí mismo.

Tabla 8. MENSAJE

Almacena los mensajes intercambiados entre estudiantes.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_mensaje	Entero	PK	Obligatorio, autogenerado	Identificador único del mensaje.
id_emisor	Entero	FK	Obligatorio	Estudiante que envía el mensaje.
id_receptor	Entero	FK	Obligatorio	Estudiante que recibe el mensaje.
texto	Texto largo	-	Obligatorio	Contenido del mensaje.
estado	Texto	-	Obligatorio	Estado del mensaje, por

				ejemplo, enviado o leído.
fecha_envio	Fecha y hora	-	Obligatorio	Fecha y hora de envío.

id_emisor e **id_receptor** hacen referencia a **ESTUDIANTE**. Se recomienda definir los estados permitidos para evitar valores inconsistentes.

Tabla 9. GRUPO

Contienge los grupos creados para reunir estudiantes alrededor de temas de interés.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_grupo	Entero	PK	Obligatorio, autogenerado	Identificador único del grupo.
nombre	Texto	-	Obligatorio	Nombre del grupo.
tema	Texto	-	Obligatorio	Tema principal del grupo.
descripcion	Texto largo	-	Opcional	Descripción y propósito del grupo.

Tabla 10. MIEMBRO_GRUPO

Relaciona a los estudiantes con los grupos a los que pertenecen.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_estudiante	Entero	PK, FK	Obligatorio	Estudiante que pertenece al grupo.
id_grupo	Entero	PK, FK	Obligatorio	Grupo al que pertenece.
rol	Texto	-	Obligatorio	Rol del estudiante dentro del grupo.
fecha_registro	Fecha y hora	-	Obligatorio	Fecha en que se unió al grupo.

La clave primaria compuesta evita que un estudiante aparezca varias veces como miembro del mismo grupo.

Tabla 11. EVENTO

Almacena los eventos organizados por los grupos de la red social.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_evento	Entero	PK	Obligatorio, autogenerado	Identificador único del evento.
id_grupo	Entero	FK	Obligatorio	Grupo organizador del evento.
nombre	Texto	-	Obligatorio	Nombre del evento.
fecha	Fecha y hora	-	Obligatorio	Fecha y hora programada para el evento.
lugar	Texto	-	Obligatorio	Lugar donde se realizará
descripcion	Texto largo	-	Opcional	Información adicional del evento.

La clave foránea **id_grupo** relaciona cada evento con el grupo que lo organiza, de acuerdo con el modelo lógico.

Tabla 12. ASISTENTE_EVENTO

Registra la participación de los estudiantes en los eventos.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_estudiante	Entero	PK, FK	Obligatorio	Estudiante asociado al evento.
id_evento	Entero	PK, FK	Obligatorio	Evento al que se asocia.
estado	Texto	-	Obligatorio	Estado de participación, por ejemplo, inscrito o asistió.
fecha_registro	Fecha y hora	-	Obligatorio	Fecha en que se registró la participación.

La clave primaria compuesta evita registrar varias veces al mismo estudiante para el mismo evento.

Tabla 13. PUBLICACION

Almacena las publicaciones realizadas por los estudiantes.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_publicacion	Entero	PK	Obligatorio, autogenerado	Identificador único de la publicación.
id_estudiante	Entero	FK	Obligatorio	Estudiante que realiza la publicación.
contenido	Texto largo	-	Obligatorio	Contenido de la publicación.
fecha	Fecha y hora	-	Obligatorio	Fecha y hora de publicación.
tipo	Texto	-	Obligatorio	Tipo de publicación, por ejemplo, texto o imagen.

El atributo **id_estudiante** identifica al autor de la publicación.

Tabla 14. COMENTARIO

Guarda los comentarios que los estudiantes realizan sobre las publicaciones.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_comentario	Entero	PK	Obligatorio, autogenerado	Identificador único del comentario.
id_publicacion	Entero	FK	Obligatorio	Publicación a la que pertenece.
id_estudiante	Entero	FK	Obligatorio	Estudiante que escribe el comentario.
contenido	Texto largo	-	Obligatorio	Texto del comentario.
fecha	Fecha y hora	-	Obligatorio	Fecha y hora de creación

				del comentario.

Las claves foráneas permiten identificar tanto la publicación comentada como el estudiante autor del comentario.

Tabla 15. REACCION

Registra las reacciones de los estudiantes a las publicaciones.

Atributo	Tipo genérico	Clave	Restricción propuesta	Descripción
id_reaccion	Entero	PK	Obligatorio, autogenerado	Identificador único de la reacción.
id_publicacion	Entero	FK	Obligatorio	Publicación que recibe la reacción.
id_estudiante	Entero	FK	Obligatorio	Estudiante que realiza la reacción.
tipo_reaccion	Texto	-	Obligatorio	Tipo de reacción, por ejemplo, me gusta.
fecha	Fecha y hora	-	Obligatorio	Fecha y hora en que se registró.

Se recomienda definir los tipos de reacción permitidos. Si el diseño establece que cada estudiante solo puede reaccionar una vez a cada publicación, deberá agregarse una restricción de unicidad sobre **id_publicacion** e **id_estudiante.**

Implementación física en MySQL

Tipo MySQL	Uso propuesto
INT	Identificadores númericos y semestre.
VARCHAR(n)	Nombres, correos, estados, roles y otros textos cortos.
TEXT	Biografías, descripciones y contenido extenso.
DATE	Fechas sin hora.
DATETIME	Fechas y hora de registro, envío o publicación.
JSON	Información semiestructurada de sensores u otros datos variables.

Implementación física en MongoDB

Colección MongoDB	Representación
estudiantes	Datos principales del estudiante.
perfiles	Perfil vinculado al estudiante.
habilidades	Catálogo de habilidades.
estudiante_habilidad	Relación entre estudiantes y habilidades.
intereses	Catálogo de intereses.
estudiante_intereses	Relación entre estudiantes e intereses.
seguir	Seguimientos entre estudiantes.
mensajes	Mensajes con emisor y receptor.
grupos	Información de los grupos.
miembro_grupo	Miembros y roles dentro de los grupos.
eventos	Eventos organizados por los grupos.
asistente_evento	Inscripciones y asistentes.
publicaciones	Publicaciones de los estudiantes.
comentarios	Comentarios asociados a publicaciones.
reacciones	Reacciones asociadas a publicaciones.

Índices, restricciones y observaciones

- MySQL: correo_institucional es único; PERFIL.id_estudiante es único para permitir como máximo un perfil por estudiante. Las tablas de relación usan claves primarias compuestas y claves foráneas.
- MySQL: SEGUIR debe impedir que un estudiante se siga a sí mismo mediante CHECK (id_seguidor <> id_seguido). Las restricciones CHECK requieren una versión moderna de MySQL 8.x para ser aplicadas.
- MySQL: las fechas de creación o registro que tengan DEFAULT CURRENT_TIMESTAMP se asignan automáticamente cuando no se entrega una fecha.
- MongoDB: se recomienda crear índices únicos para correo_institucional, perfiles.id_estudiante y pares de campos de las colecciones de relación: estudiante_habilidad, estudiante_intereses, seguir, miembro_grupo y asistente_evento.
- MongoDB: las referencias id_* deben contener el ObjectId real del documento referenciado. MongoDB no impone claves foráneas; esa verificación corresponde a la lógica de la aplicación.

- MongoDB: si se ejecuta el script en una base que ya tiene colecciones, confirmar que los validadores e índices coincidan con este diccionario; crear colecciones existentes no actualiza automáticamente su validador.
- El modelo agrega datos_adicionales en EVENTO para justificar datos semiestructurados relacionados con Big Data o IoT. Puede guardar coordenadas, temperatura, humedad, código de sensor y marca de tiempo.

4. Ejemplo del campo semiestructurado datos_adicionales

```
{
  "sensor_id": "sensor-aula-03",
  "coordenadas": { "latitud": 4.628, "longitud": -74.064 },
  "temperatura_c": 24.6,
  "humedad_pct": 61,
  "lectura_fecha": "2026-10-25T14:00:00Z"
}
```

Ejemplo ilustrativo; el sistema puede guardar otras propiedades en función del evento o del dispositivo que genere los datos.

Implementación física en PostgreSQL

Tipo genérico	Tipo en PostgreSQL	Uso
Entero (PK autogenerada)	INTEGER GENERATED ALWAYS AS IDENTITY	Todas las claves id_*
Entero	SMALLINT	semestre
Texto	VARCHAR(n)	Nombres, correo, estados, roles
Texto largo	TEXT	biografía, contenido, descripción
Fecha	DATE	fecha_aprendizaje
Fecha y hora	TIMESTAMP (DEFAULT CURRENT_TIMESTAMP)	Fechas de registro, envío y publicación
Semiestructurado	JSONB	datos_sensores, metadatos

Decisiones adicionales al pasar de lo genérico a lo físico:

- Se corrigieron dos errores del genérico: id_publiacion → id_publicacion y la clave PF → PK en ESTUDIANTE_HABILIDAD.
- Los campos de estado, rol, nivel y tipo se limitaron con CHECK para evitar valores inconsistentes.

- seguir y mensaje impiden que un estudiante se siga o se escriba a sí mismo (CHECK id_seguidor <> id_seguido, id_emisor <> id_receptor).
- perfil.id_estudiante es UNIQUE para garantizar la relación 1:1.
- reaccion tiene UNIQUE (id_publicacion, id_estudiante): un estudiante reacciona una sola vez por publicación.
- Se crearon índices sobre las FK de mayor consulta e índices GIN sobre las columnas JSONB.

Justificación:

- Los dispositivos IoT envían estructuras distintas según el tipo de sensor; con columnas fijas habría que alterar el esquema cada vez que se agregue una medición.
- En big data los metadatos crecen en volumen y variedad; JSONB los absorbe sin cambiar la tabla.
- Se eligió JSONB sobre JSON porque se guarda en binario, admite índices GIN y se consulta más rápido.
- Un CHECK con jsonb_typeof(...) = 'object' asegura que el valor guardado sea un objeto JSON válido.

Relaciones y acciones referenciales

Todas las claves foráneas usan **ON UPDATE CASCADE** y **ON DELETE CASCADE**.

Relación	Justificación del ON DELETE CASCADE
estudiante → perfil, publicacion, mensaje, seguir, estudiante_habilidad, estudiante_interes, miembro_grupo, asistente_evento	Si se elimina un estudiante, sus datos dependientes pierden sentido
publicacion → comentario, reaccion	Un comentario o reacción no existe sin su publicación
grupo → evento, miembro_grupo	Los eventos pertenecen al grupo que los organiza
evento → asistente_evento	La asistencia depende del evento

habilidad, interes → tablas intermedias	Se elimina solo la asociación, no al estudiante

Las relaciones N:M (estudiante_habilidad, estudiante_interes, seguir, miembro_grupo, asistente_evento) se resolvieron con claves primarias compuestas, lo que evita registros duplicados.

Conclusiones individuales

Juan David Jovel: Durante el desarrollo de este proyecto pude comprender mejor cómo se lleva un modelo lógico a una implementación física y cómo cada sistema gestor tiene sus propias características y formas de trabajar. También aprendí la importancia de definir correctamente los tipos de datos, las relaciones y las restricciones para mantener la información organizada. Considero que comparar MySQL y MongoDB fue una experiencia útil, porque me permitió conocer las diferencias entre las bases de datos relacionales y las documentales, así como las posibilidades que ofrece el manejo de datos semiestructurados para futuras aplicaciones.

Angie Vanessa Rincón: Implementar el modelo en PostgreSQL me mostró que diseñar una base de datos en papel es muy distinto a hacerla funcionar. Al pasarla a la práctica tuve que decidir cosas concretas, como qué tipo de dato usar en cada campo, qué valores se permiten y qué pasa con la información relacionada cuando se borra un registro.

Lo que más me gustó fue ver que PostgreSQL permite guardar datos ordenados y también datos más flexibles, como los de los sensores o la información extra de las publicaciones, todo en la misma base. Así aprendí que no siempre hay que cambiar de herramienta cuando los datos no son tan uniformes.

Además, comparar mi trabajo con el de Juan David Jovel López, que lo hizo en MySQL y MongoDB, me enseñó que una misma idea se puede construir de

formas distintas, y que lo importante es elegir la herramienta que mejor se ajuste a lo que necesita el proyecto.